Yitao’s Work summary at Galbot

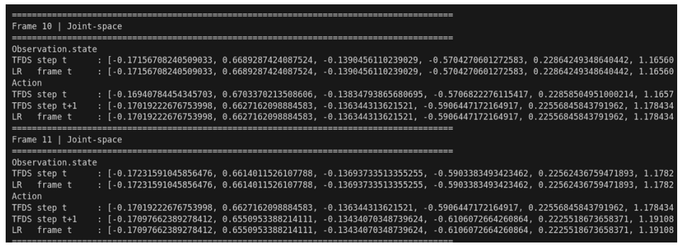
4.25 Beijing



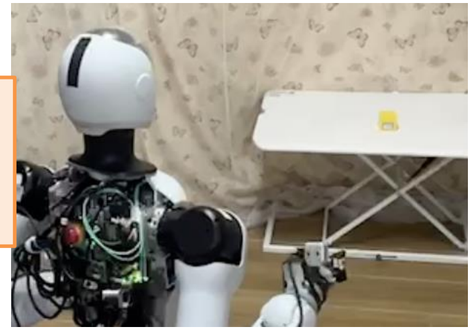
Research Roadmap of the Past 2 Months

Baseline implementation on a large-scale simulation dataset





Conversion of two types of datasets(TFDS,LeRobot)



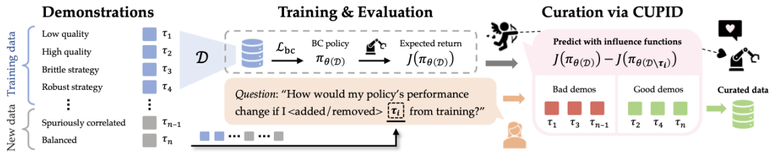
Real robot testing on foundation models

Exploration of dataset influence on models

CUPID: Curating Data your Robot Loves with Influence Functions

Christopher Agia[1], Rohan Sinha[1], Jingyun Yang[1], Rika Antonova[2], Marco Pavone[1,3], Haruki Nishimura[4], Masha Itkina[4], Jeannette Bohg[1]

[1]Stanford University, [2]University of Cambridge, [3]NVIDIA Research, [4]Toyota Research Institute

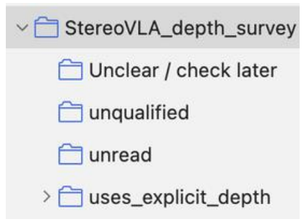


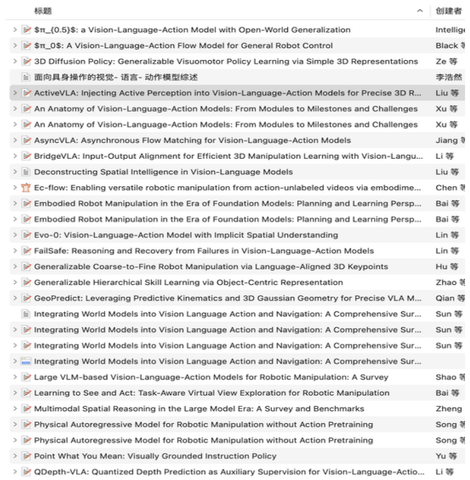
1.Baseline for Stereo-VLA, 3D diffusion policy

Literature Review → Baseline 3D Diffusion Policy

Surveying

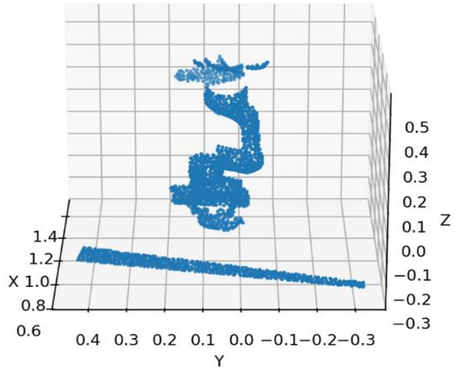
1. Explicit depth info
2. Renown and published
3. Franka arm



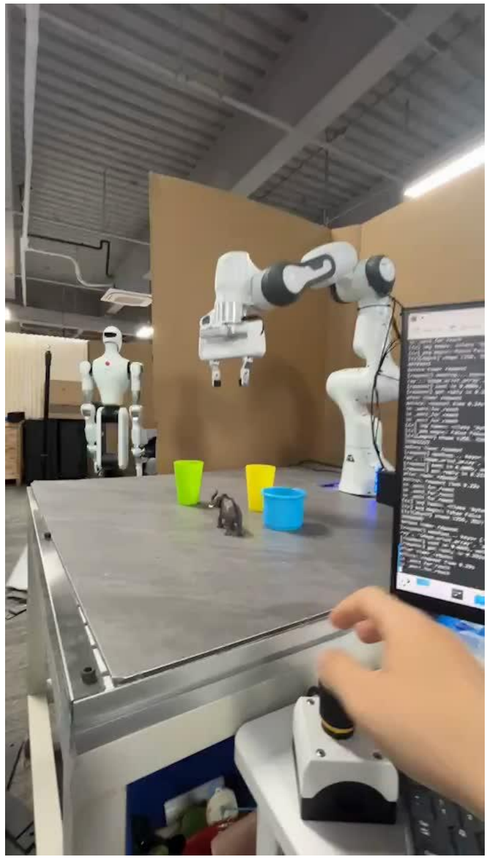


Dataset pick-elephant-sim (~6M) → Online Streaming Preprocessing → Depth → Point Cloud Axis-aligned Cropping (FPS = 1024) → Training → Inference → Franka ROS Controller

TFDS data processing



Used streaming to save memory



Could run but couldn’t pick up

Failure analysis

1.Sim2real gap(for pointclouds)

2.Too much noise in pointclouds (required no calibration)

Difficulties:

1.Complex key-pair matching(quat,abs/delta)

2.Streaming the point cloud processing during training

3.Docker deploy on franka

Learnings:

1.The debugging process of tensor sizes

2.Debugging techniques with AI

3.Ability to integrate original repo's APIs with tasks I want to do.

Example during key matching

dataset:

command_eef_orientation：dim：4，wxyz

command_eef_position:dim:3

eef_position:dim3

eef_orientation:dim4

gripper_proprio：-1，1（只会在最后一位出现0）

gripper_action: -1,,1（只会在最后一位出现0）

关于gripper是否有0还是不是非常清晰，不过应该是能够对应上的。

deploy:

Client to server:

text, image_wrist_array, image_array, depth_array, proprio_array, env_id, traj_metadata, compressed.

proprio_array:dim7:xyzrpygripper

gripper:1 if open and -1 if closed

2.TFDS to LeRobot dataset transfer

TFDS Dataset:
Mem:20TB
Episodes:5mil



Step1:

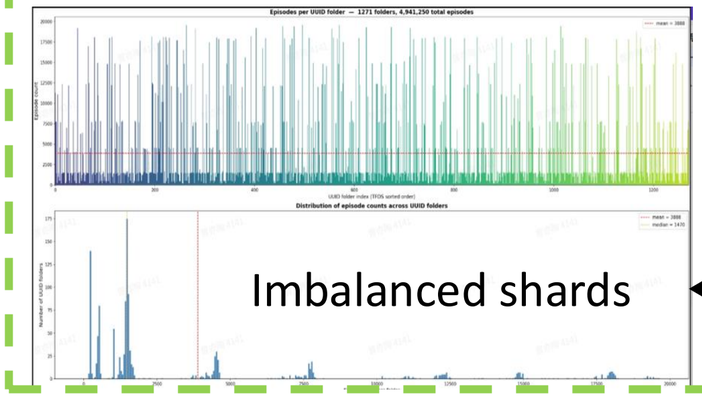
Key mapping → Few episode validation

```
================================================================================
Frame 10 | Joint-space
================================================================================
Observation.state
TFDS step t      : [-0.17156708240509033, 0.6689287424087524, -0.1390456110239029, -0.5704270601272583, 0.22864249348640442, 1.16560
LR   frame t     : [-0.17156708240509033, 0.6689287424087524, -0.1390456110239029, -0.5704270601272583, 0.22864249348640442, 1.16560
Action
TFDS step t      : [-0.16940784454345703, 0.6703370213508606, -0.13834793865680695, -0.5706822276115417, 0.22858504951000214, 1.1657
TFDS step t+1    : [-0.17019222676753998, 0.6627162098884583, -0.136344313621521, -0.5906447172164917, 0.22556845843791962, 1.178434
LR   frame t     : [-0.17019222676753998, 0.6627162098884583, -0.136344313621521, -0.5906447172164917, 0.22556845843791962, 1.178434
================================================================================
Frame 11 | Joint-space
================================================================================
Observation.state
TFDS step t      : [-0.17231591045856476, 0.6614011526107788, -0.13693733513355255, -0.5903383493423462, 0.22562436759471893, 1.1782
LR   frame t     : [-0.17231591045856476, 0.6614011526107788, -0.13693733513355255, -0.5903383493423462, 0.22562436759471893, 1.1782
Action
TFDS step t      : [-0.17019222676753998, 0.6627162098884583, -0.136344313621521, -0.5906447172164917, 0.22556845843791962, 1.178434
TFDS step t+1    : [-0.17097662389278412, 0.6550953388214111, -0.13434070348739624, -0.6106072664260864, 0.2225518673658371, 1.19108
LR   frame t     : [-0.17097662389278412, 0.6550953388214111, -0.13434070348739624, -0.6106072664260864, 0.2225518673658371, 1.19108
================================================================================
```

Step2:

Parallel conversion → Optimization

- CPU: AMD/Intel
- Num of CPUs per card
- Use parallel encoding or not
- Num of threads
- Num of world size per card
- Num of cards
- Different variations of codes



Imbalanced shards

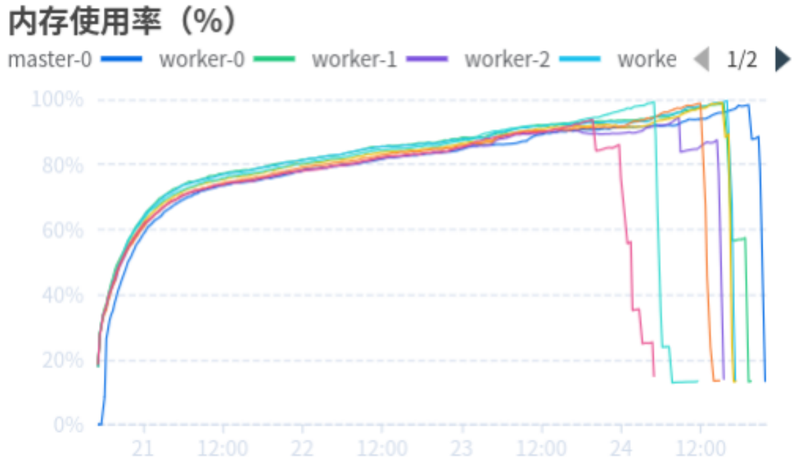
Restructure the dataset → Inverse order

Conversion

Local world size:9
Card:25
CPU:Intel
Thread:12
Result:
4d4h53min
Failure rate:0.46%
Mem:4.91TB

Difficulties:

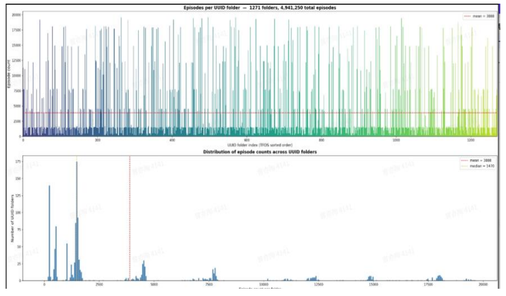
1. Memory growing.
2. Increasing speed.
3. Restructuring dataset.
4. Parameter optimization.



Learnings:

1. Usage of plotting graphs to support understanding.
2. Planning experiments is more important than brute testing.

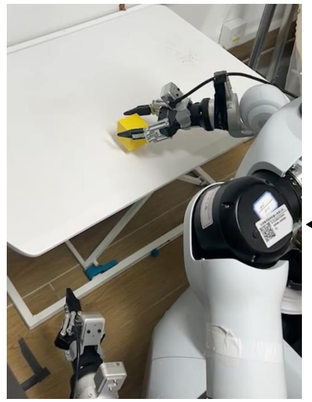
Unnecessary 67 attempts



Tfds dataset per shard distribution

3.Foundation model real robot experiment

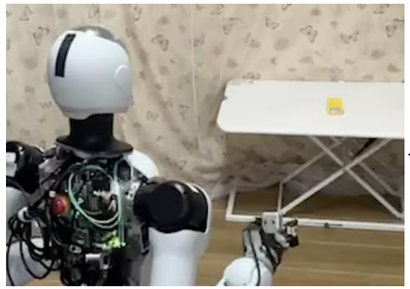
Static chassis: Extracting initial pose from simulated environments to achieve high success rate



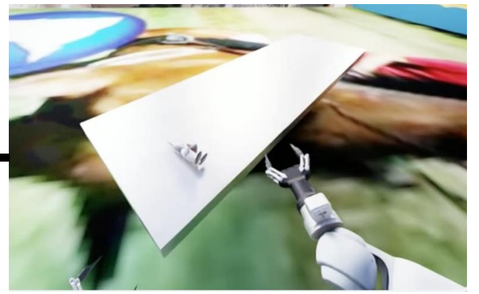
High success rate

1:手臂低于桌面，危险（尝试两次）
2：成功率高，但位置有点奇怪
手动搬了一个看似不错的位置，但是失败
3：手腕关节打到自己身上，而且速度很快
4：成功，位置有点奇怪，但还算不错的了
5：成功，位置很低，但是成功
6：第一次失败，起始位置低，扫过去时向下压物块，崩飞
7：跟2动作相似，没有测
8：动作跟4像，成功
9：非常危险的动作，差点打到自己的胸了
10：轴关节非常低，低于桌面

Tested 10 groups



Extract



Mobile chassis: Fixed and verified chassis controller

New functions https://git.galbot.com/sunbowen/galbot_client/-/tree/zyt?ref_type=heads

This branch is pulled from main

Plot the robot trajectory of chassis:

```
uv run python src/galbot_client/plot_chassis_traj.py --file-path
```

Chassis orientation:

（+：前，+：左，+逆时针）eg:'chassis': array([-0.0165127 , 0.01722821, 0.04805565])

Recording videos and logs in /logs

Eg: the dummy server:

```
uv run python -m galbot_client.client --host 127.0.0.1 --port 8765 --action-steps 1 --save-video --video-fps 10
```

Note that there's still a slight error of the controller, it usually executes the action a bit smaller(in angle and length) than the command.

1. Automatically plot traj
2. Automatic logs and video recording

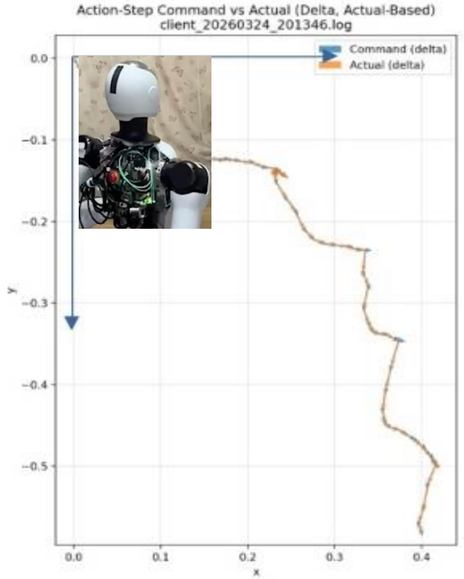


Table is here



Drifted away, but aligns partly wth sim rollouts!



Dummy server to verification

Forward： 1m

Turn right： 45 degrees

Forward ： 1m

Turn left： 45 degrees

Failure:

1. The room is too different
2. The curtains looked like the object
3. The table in sim is mostly long and narrow
4. Similar failure trait in simulator

4.Post-training dataset optimization

Why we need data optimization for post-training?

1. Prevent model overfitting and boost generalization.
2. Make training computationally efficient.
3. Lower the cost of creating dataset(like labeling)

When exactly should optimization happen?

1. During collecting data
2. During processing of an already collected dataset

What are potential flaws of a post-training dataset?

1. Irrelevant data(noise)
2. Imbalanced data proportion.
3. Failed trajectories.



All about the quality!

How to define the quality of a data to a specific task?

1.Heuristically (human defined)

2.Standard measurements:

3.Result based

4.Loss based

1. Wang & Strong (1996): *Beyond Accuracy: What Data Quality Means to Data Consumers*

Accuracy	Data correctly reflects reality.
Completeness	All required information present.
Consistency	No contradictions across records.
Timeliness/Currency	Up-to-date relative to use.
Believability / Credibility	Users trust the source.
Relevance / Contextual fit	Data useful for a given task.
Interpretability / Representational clarity	Easy for humans/systems to parse and use.
Accessibility	Usable when and where needed.

2.

Robot Data Curation
with Mutual Information Estimators

Joey Hejna[1,2,*], Suvir Mirchandani[2], Ashwin Balakrishna[1], Annie Xie[1], Ayzaan Wahid[1], Jonathan Tompson[1], Pannag Sanketi[1], Dhruv Shah[1], Coline Devin[1,†], Dorsa Sadigh[1,†]
[1]Google DeepMind Robotics, [2]Stanford University

Influence functions

Understanding Black-box Predictions via Influence Functions

作者：PW Koh · 2017 · 被引用次数：4539 — In this paper, we use influence functions -

Train on Validation (ToV):
Fast data selection with applications to fine-tuning

How to define the quality of a data to a specific task?

1. Heuristically (human defined)

2. Standard measurements:

3. Result based

4. Loss based

1. Wang & Strong (1996): *Beyond Accuracy: What Data Quality Means to Data Consumers*

Accuracy	Data correctly reflects reality.
Completeness	All required information present.
Consistency	No contradictions across records.
Timeliness/Currency	Up-to-date relative to use.
Believability / Credibility	Users trust the source.
Relevance / Contextual fit	Data useful for a given task.
Interpretability / Representational clarity	Easy for humans/systems to parse and use.
Accessibility	Usable when and where needed.

Influence functions

2. Robot Data Curation with Mutual Information Estimators

Joey Hejna[1,2,*], Suvir Mirchandani[2], Ashwin Balakrishna[1], Annie Xie[1], Ayzaan Wahid[1], Jonathan Tompson[1], Pannag Sanketi[1], Dhruv Shah[1], Coline Devin[1,†], Dorsa Sadigh[1,†]
[1]Google DeepMind Robotics, [2]Stanford University

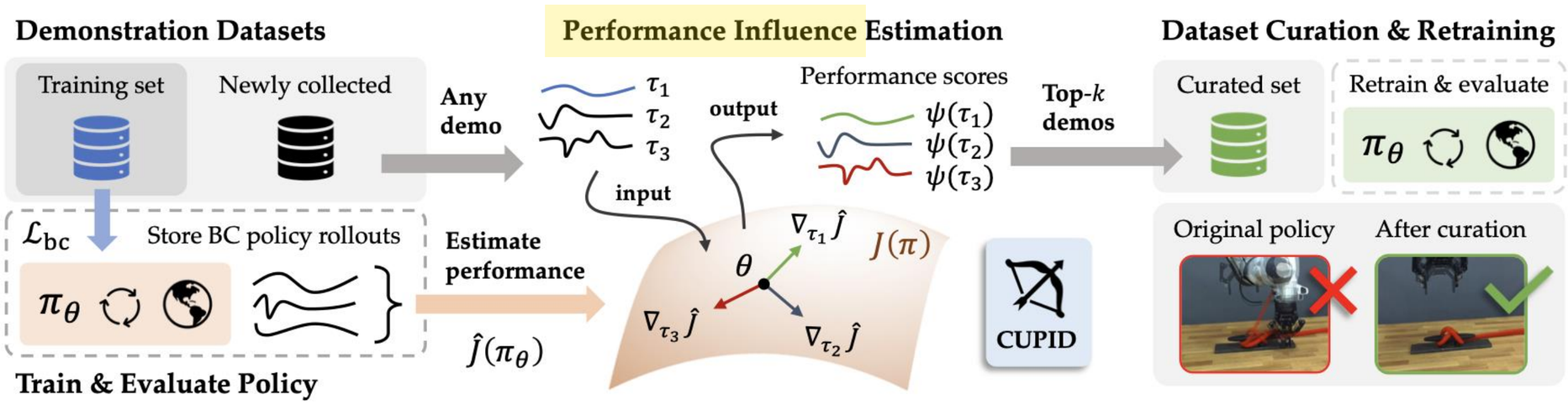
Only this has a casual link to the model's actual behavior

Train on Validation (ToV):
Fast data selection with applications to fine-tuning

Beyond Heuristics: Influence Functions for Robot Data Curation

Yitao Zeng 4.21

SUSTech



Beyond Heuristics: Influence Functions for Robot Data Curation

Yitao Zeng 4.21

SUSTech

Model group's weekly presentation

Influence functions:

Counterfactual: “What if”

- What if I took this demonstration away?
- What if I added this trajectory?

Mathematical expression:

Ψ The influence

$$\Psi^{\text{out}}(\hat{z},z^i) \approx f\left(\hat{z};\theta(\mathcal{D}\setminus\{z^i\})\right) - f(\hat{z};\theta(\mathcal{D})).$$

$$\Psi^{\text{in}}_{\text{inf}}(\hat{z},z) := \frac{1}{n}\Psi_{\text{inf}}(\hat{z},z) \approx f(\hat{z};\theta(\mathcal{D}\cup\{z\})) - f(\hat{z};\theta(\mathcal{D}))$$

The impact in ML

Perturbed training objective

$$\mathcal{L}_{\epsilon,z}(\theta; D) = \mathcal{L}(\theta; D) + \epsilon\ell(z;\theta)$$

$$\hat{\theta}_{\epsilon,z} := \arg\min_{\theta}\left(\mathcal{L}(\theta; D) + \epsilon\,\ell(z;\theta)\right)$$

Without retraining!

$$\Psi_{\text{inf}}(\hat{z},z) := \left.\frac{df(\hat{z};\theta)}{d\epsilon}\right|_{\epsilon=0} = -\nabla_\theta f(\hat{z};\theta(\mathcal{D}))^\top H_\theta^{-1}\nabla_\theta\ell(z;\theta(\mathcal{D})),$$

f: performance metric

z: actual training demo

$H_\theta = \frac{1}{n}\sum_{i=1}^{n}\nabla_\theta^2\ell(z^i;\theta(\mathcal{D}))$ denotes the Hessian of the training loss

$\hat{z}$:test demo

Influence functions:

Counterfactual: “What if”

- What if I took this demonstration away?
- What if I added this trajectory?

?

Mathematical expression:

Ψ The influence

$$\Psi^{\text{out}}(\hat{z},z^i) \approx f\left(\hat{z};\theta(\mathcal{D}\setminus\{z^i\})\right) - f(\hat{z};\theta(\mathcal{D})).$$

$$\Psi^{\text{in}}_{\text{inf}}(\hat{z},z) := \frac{1}{n}\Psi_{\text{inf}}(\hat{z},z) \approx f(\hat{z};\theta(\mathcal{D}\cup\{z\})) - f(\hat{z};\theta(\mathcal{D}))$$

The impact in ML

Perturbed training objective

$$\mathcal{L}_{\epsilon,z}(\theta; D) = \mathcal{L}(\theta; D) + \epsilon\ell(z;\theta)$$

$$\hat{\theta}_{\epsilon,z} := \arg\min_{\theta}\left(\mathcal{L}(\theta; D) + \epsilon\,\ell(z;\theta)\right)$$

$$\Psi_{\text{inf}}(\hat{z},z) := \left.\frac{df(\hat{z};\theta)}{d\epsilon}\right|_{\epsilon=0} = -\nabla_\theta f(\hat{z};\theta(\mathcal{D}))^\top H_\theta^{-1}\nabla_\theta\ell(z;\theta(\mathcal{D})),$$

f: performance metric

z: actual training demo

$H_\theta = \frac{1}{n}\sum_{i=1}^{n}\nabla_\theta^2\ell(z^i;\theta(\mathcal{D}))$ denotes the Hessian of the training loss

$\hat{z}$:test demo

More about math

$$\hat{\theta}_{\epsilon,z} = \arg\min_{\theta\in\Theta} \{R(\theta) + \epsilon L(z,\theta)\}.$$

$$0 = \nabla R(\hat{\theta}_{\epsilon,z}) + \epsilon \nabla L(z, \hat{\theta}_{\epsilon,z}).$$

Next, since $\hat{\theta}_{\epsilon,z} \to \hat{\theta}$ as $\epsilon \to 0$, we perform a Taylor expansion of the right-hand side:

$$0 \approx \left[\nabla R(\hat{\theta}) + \epsilon \nabla L(z,\hat{\theta})\right] + \left[\nabla^2 R(\hat{\theta}) + \epsilon \nabla^2 L(z,\hat{\theta})\right] \Delta_\epsilon, \tag{11}$$

where we have dropped $o(\|\Delta_\epsilon\|)$ terms.

Solving for Δ_ϵ, we get:

$$\Delta_\epsilon \approx -\left[\nabla^2 R(\hat{\theta}) + \epsilon \nabla^2 L(z,\hat{\theta})\right]^{-1} \left[\nabla R(\hat{\theta}) + \epsilon \nabla L(z,\hat{\theta})\right]. \tag{12}$$

Since $\hat{\theta}$ minimizes R, we have $\nabla R(\hat{\theta}) = 0$. Dropping $o(\epsilon)$ terms, we have

$$\Delta_\epsilon \approx -\nabla^2 R(\hat{\theta})^{-1} \nabla L(z,\hat{\theta})\epsilon. \tag{13}$$

Combining with (7) and (9), we conclude that:

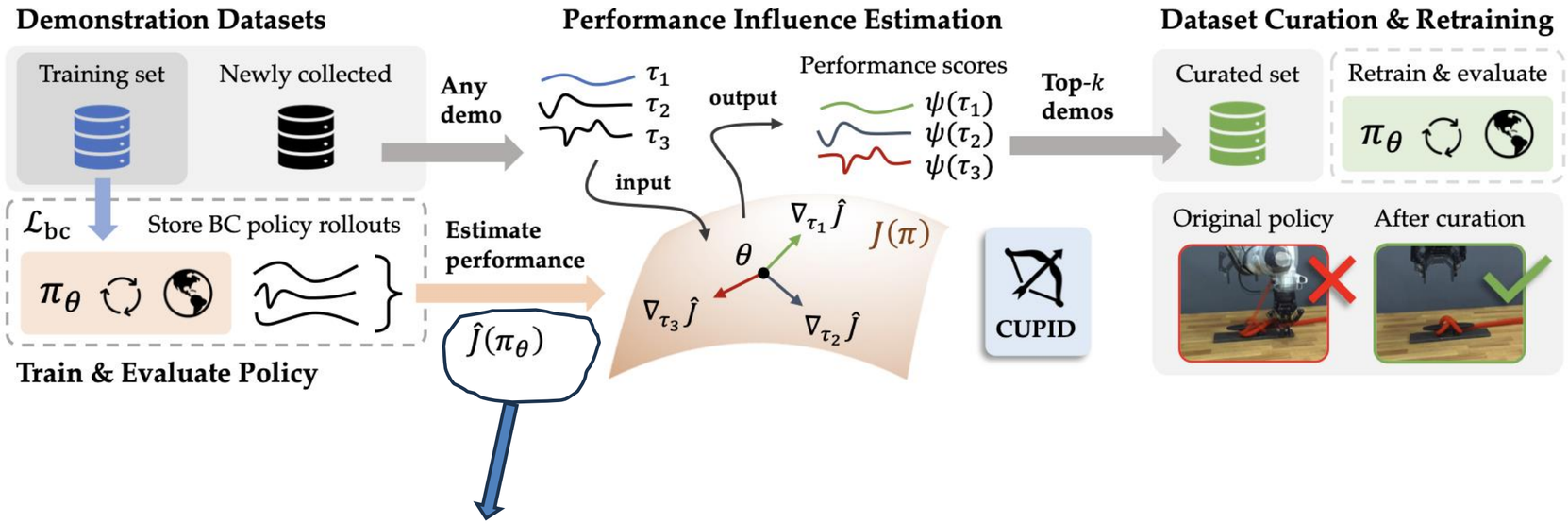
$$\left.\frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon}\right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla L(z,\hat{\theta}) \tag{14}$$

$$\stackrel{\text{def}}{=} \mathcal{I}_{\text{up,params}}(z). \tag{15}$$

CUPID: Curating Data your Robot Loves with Influence Functions

Christopher Agia[1], Rohan Sinha[1], Jingyun Yang[1],
Rika Antonova[2], Marco Pavone[1,3], Haruki Nishimura[4], Masha Itkina[4], Jeannette Bohg[1]
[1]Stanford University, [2]University of Cambridge, [3]NVIDIA Research, [4]Toyota Research Institute

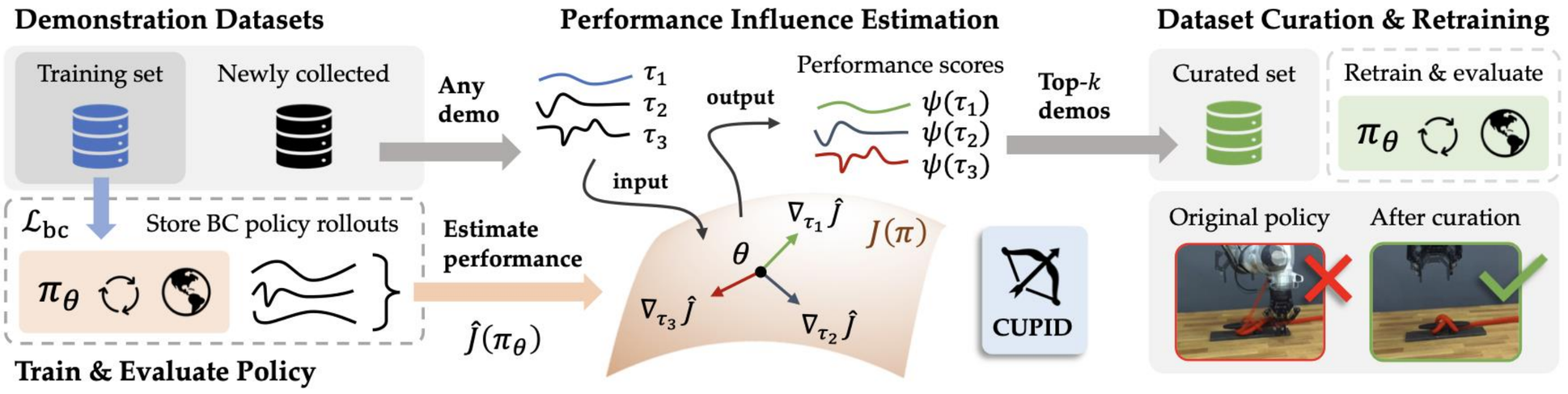
First to use influence functions on robotic datasets

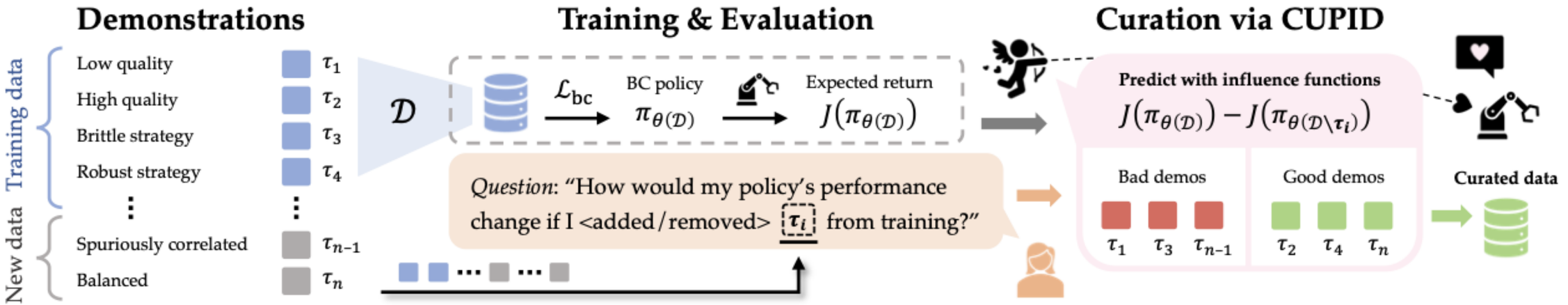


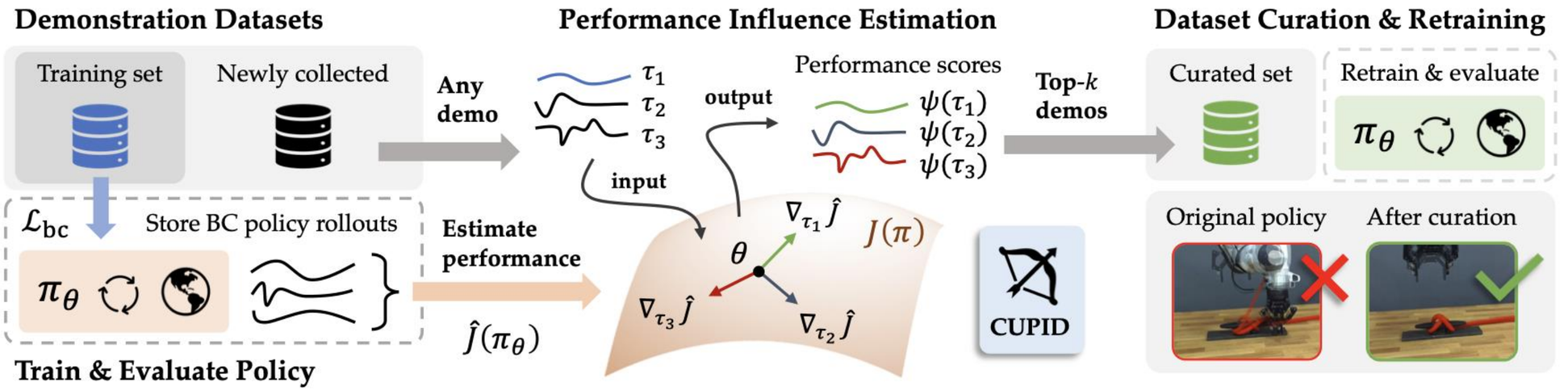
$$\Psi_{\text{inf}}(\hat{z},z) := \left.\frac{df(\hat{z};\theta)}{d\epsilon}\right|_{\epsilon=0} = -\nabla_\theta f(\hat{z};\theta(\mathcal{D}))^\top H_\theta^{-1} \nabla_\theta \ell(z;\theta(\mathcal{D})),$$

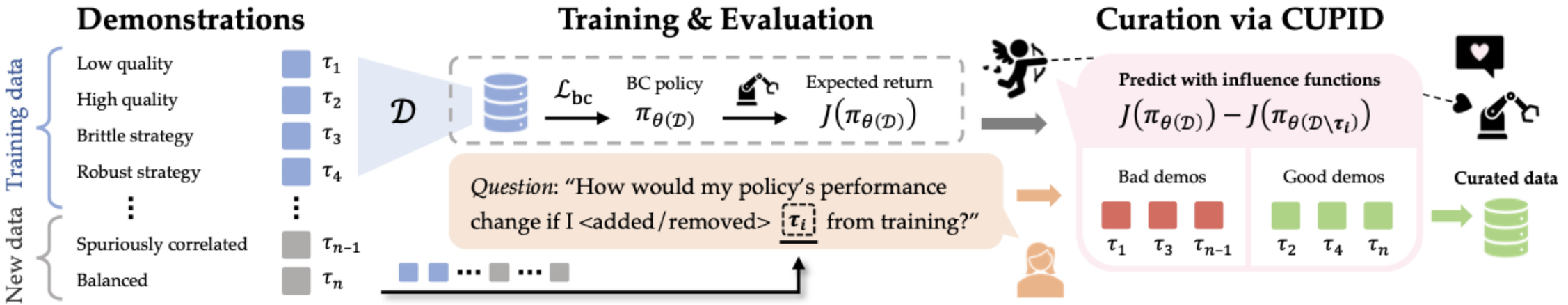
$$\Psi_{\pi\text{-inf}}(\xi) := \left.\frac{dJ(\pi_\theta)}{d\epsilon}\right|_{\epsilon=0} = -\nabla_\theta J(\pi_\theta)^\top H_{\text{bc}}^{-1} \nabla_\theta \ell_{\text{traj}}(\xi;\pi_\theta),$$

Performance influence


Demonstration Datasets
Training set
Newly collected
$\mathcal{L}_{\text{bc}}$
Store BC policy rollouts
π_θ
Train & Evaluate Policy
Any demo
Performance Influence Estimation
τ_1
τ_2
τ_3
output
input
Performance scores
$\psi(\tau_1)$
$\psi(\tau_2)$
$\psi(\tau_3)$
Estimate performance
$\hat{J}(\pi_\theta)$
$\nabla_{\tau_1}\hat{J}$
$J(\pi)$
θ
$\nabla_{\tau_3}\hat{J}$
$\nabla_{\tau_2}\hat{J}$
CUPID
Top-k demos
Dataset Curation & Retraining
Curated set
Retrain & evaluate
π_θ
Original policy
After curation


Demonstrations
Training data
New data
Low quality
High quality
Brittle strategy
Robust strategy
Spuriously correlated
Balanced
τ_1
τ_2
τ_3
τ_4
τ_{n-1}
τ_n
$\mathcal{D}$
Training & Evaluation
$\mathcal{L}_{\text{bc}}$
BC policy
$\pi_{\theta(\mathcal{D})}$
Expected return
$J(\pi_{\theta(\mathcal{D})})$
Question: "How would my policy's performance change if I <added/removed> τ_i from training?"
Curation via CUPID
Predict with influence functions
$J(\pi_{\theta(\mathcal{D})}) - J(\pi_{\theta(\mathcal{D}\setminus\tau_i)})$
Bad demos
Good demos
τ_1 τ_3 τ_{n-1}
τ_2 τ_4 τ_n
Curated data



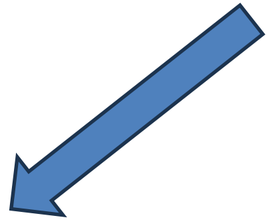


But J (Expected return) depends on the unknown transition dynamics and reward function, **how to solve it?**

Thus we use a policy gradient(REINFORCE-like) method to approximate J

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim p(\tau|\pi_\theta)}\Big[R(\tau)\sum_{(s',a')\in\tau}\nabla_\theta \log\pi_\theta(a'|s')\Big] + \Psi_{\pi\text{-inf}}(\xi) = -\nabla_\theta J(\pi_\theta)^\top H_{\text{bc}}^{-1}\nabla_\theta \ell_{\text{traj}}(\xi;\pi_\theta),$$

Policy gradient



$$\Psi_{\pi\text{-inf}}(\xi) = \mathbb{E}_{\tau\sim p(\tau|\pi_\theta)}\Big[R(\tau)\sum_{(s',a')\in\tau} -\nabla_\theta \log\pi_\theta(a'|s')^\top H_{\text{bc}}^{-1}\nabla_\theta \ell_{\text{traj}}(\xi;\pi_\theta)\Big]$$

Define as action influence

$$\Psi_{\pi\text{-inf}}(\xi) = \mathbb{E}_{\tau\sim p(\tau|\pi_\theta)}\left[\frac{R(\tau)}{H}\sum_{(s',a')\in\tau}\sum_{(s,a)\in\xi}\Psi_{a\text{-inf}}\big((s',a'),(s,a)\big)\right]$$

Where: $\Psi_{a\text{-inf}}\big((s',a'),(s,a)\big) := -\nabla_\theta \log\pi_\theta(a'|s')^\top H_{\text{bc}}^{-1}\nabla_\theta \ell(s,a;\pi_\theta).$ **Action influence**

Flaw of raw cupid: High variance

5.3 Additional Quality Metrics

In §5.1, we constructed a method to estimate $\Psi_{\pi\text{-inf}}$ from a dataset of policy rollouts $\mathcal{D}_\tau$ by relying on policy gradient methods. Therefore, the estimated performance influence $\widehat{\Psi}_{\pi\text{-inf}}$ becomes increasingly noisy as we reduce the number of rollouts m to evaluate the policy—akin to the high variance problem of the `REINFORCE` algorithm. To complement the analysis in §5.1, we explore the integration of a *reward-agnostic, heuristic* demonstration quality metric based on the action influence scores $\Psi_{a\text{-inf}}$:

$$\Psi_{\text{qual}}(\xi;\mathcal{D}_\tau) := \frac{1}{m}\sum_{\tau\in\mathcal{D}_\tau} \max_{(s',a')\in\tau} \min_{(s,a)\in\xi} \Psi_{a\text{-inf}}\big((s',a'),(s,a)\big) - \min_{(s',a')\in\tau} \max_{(s,a)\in\xi} \Psi_{a\text{-inf}}\big((s',a'),(s,a)\big). \quad (6)$$

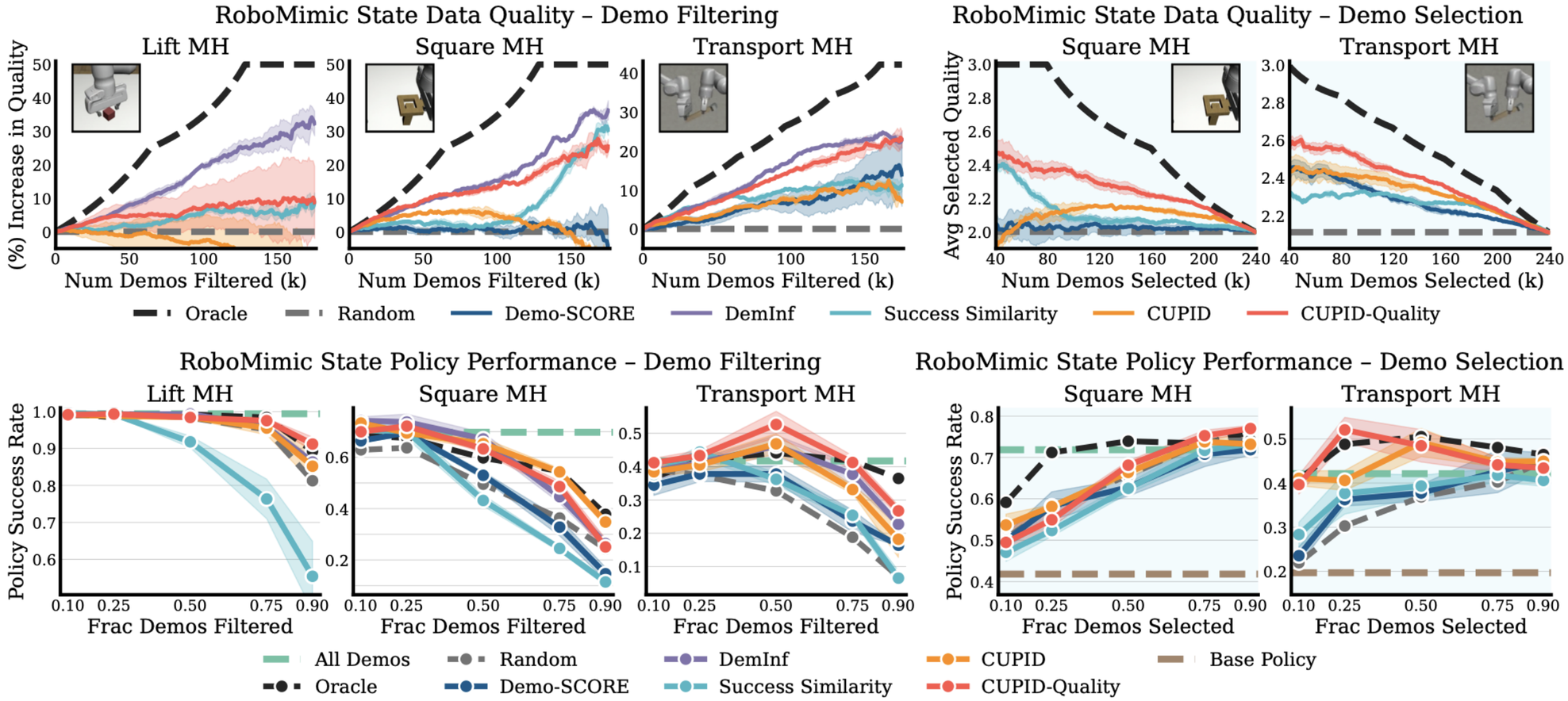
Penalize demonstrations containing outlier or noisy influence scores

Cleaner data

They call this CUPID-Quality later on

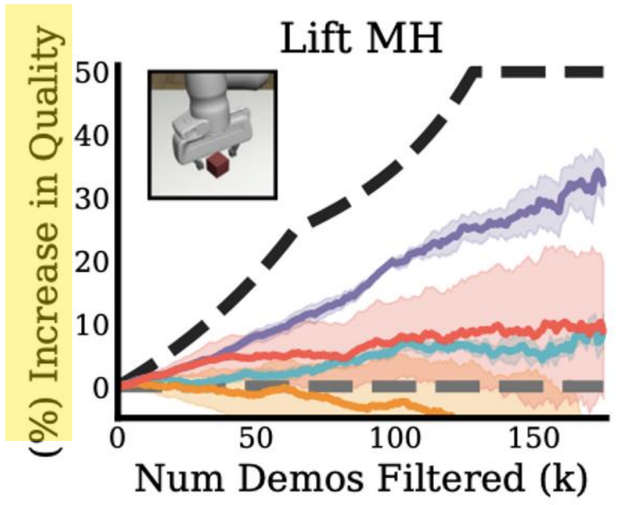
Note: the maxmin-minmax structure is not typical, however the results look good

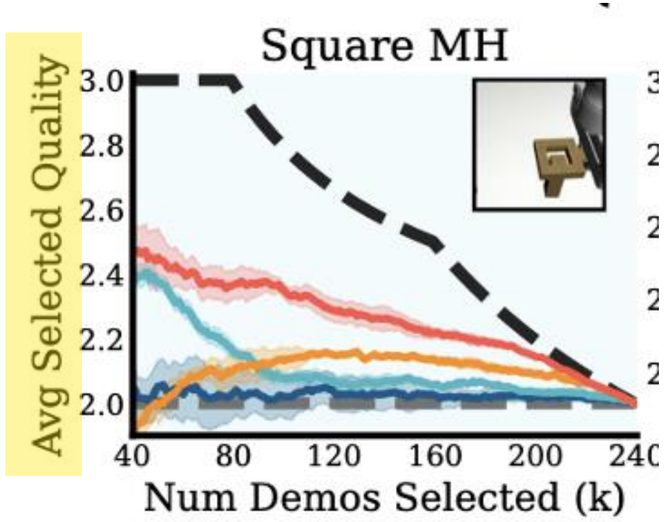
Experiments:



Sim

1. Deminf and CUPID-Quality do behave good on quality-measurement tasks
2. High quality doesn't necessarily mean good down-streaming results
3. Data with higher quality behaves good on tasks requiring high precision

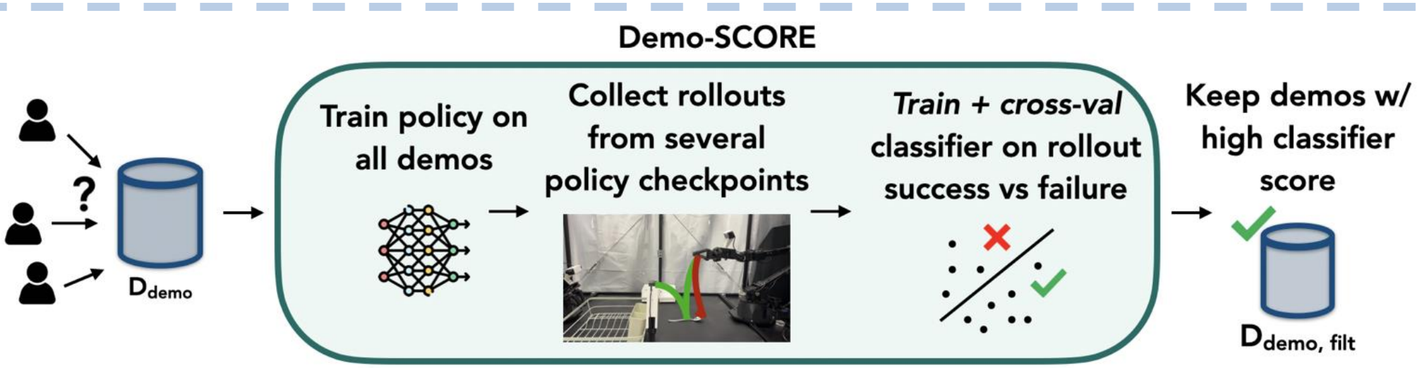




Quality here means the 3 stages of quality in robomimic

Oracle Random Demo-SCORE

curating data with privileged access to ground-truth demonstration labels



Curating Demonstrations using Online Experience

Annie S. Chen*, Alec M. Lessing*, Yuejiang Liu, Chelsea Finn
Stanford University

DemInf

Success Similarity

Mutual Information as a Quality Metric

$$I(S;A) = H(A) - H(A \mid S)$$

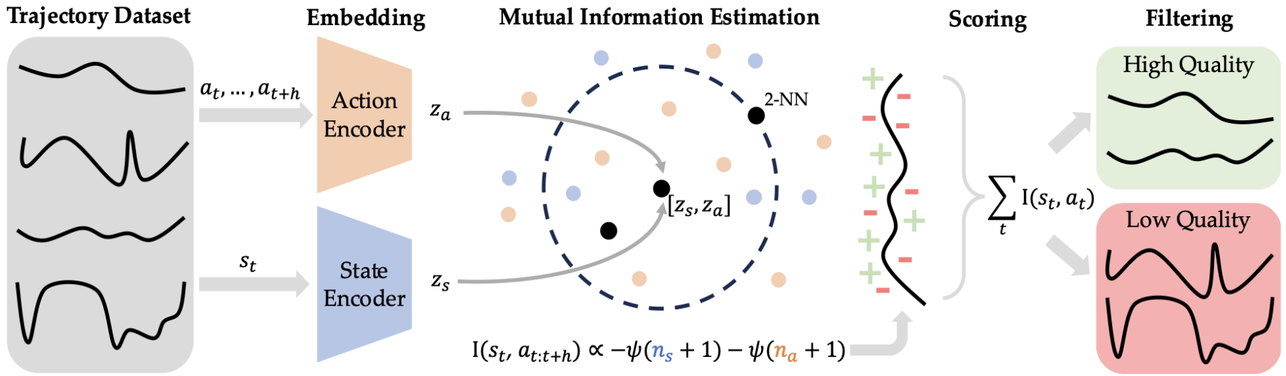


Fig. 1. A graphical depiction of the DemInf method. First, we begin by learning VAEs for states and action chunks to produce latent representations z_a and z_s. Using these latent representations, we apply the KSG k-nearest-neighbor based mutual information estimator. Finally, we filter demonstrations based on their estimated mutual information.

Data quality:

1. Action has high variety when not knowing state
2. Action is quite determined when knowing previous state

Robot Data Curation
with Mutual Information Estimators

Joey Hejna[1,2,*], Suvir Mirchandani[2], Ashwin Balakrishna[1], Annie Xie[1], Ayzaan Wahid[1], Jonathan Tompson[1], Pannag Sanketi[1], Dhruv Shah[1], Coline Devin[1,†], Dorsa Sadigh[1,†]
[1]Google DeepMind Robotics, [2]Stanford University

$$S(\xi;\mathcal{D}_\tau) = -\sum_{\tau \in \mathcal{D}_\tau} \left[\mathbf{1}(R(\tau)=1) \cdot \frac{1}{H^2} \sum_{s' \in \tau} \sum_{s \in \xi} D(\phi(s'), \phi(s)) \right],$$

D is a specified distance function over state embeddings, they wrote it themselves.

Real

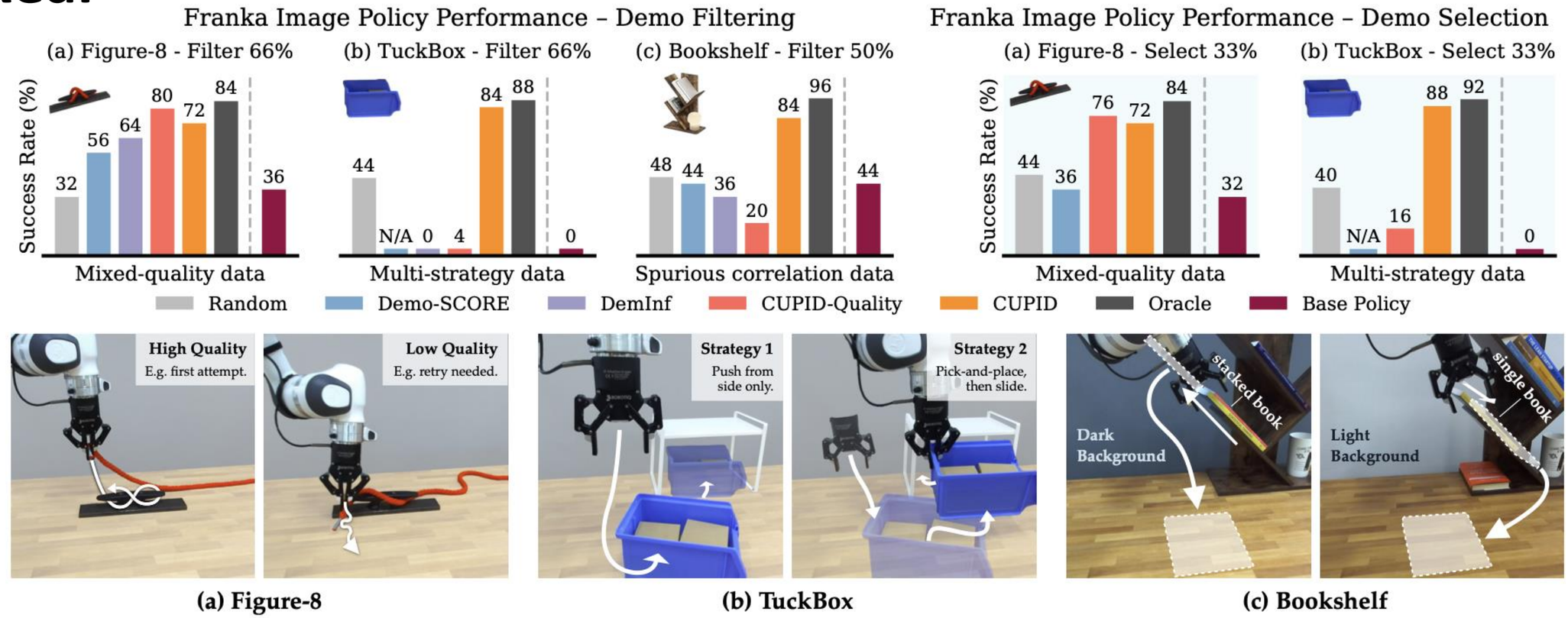


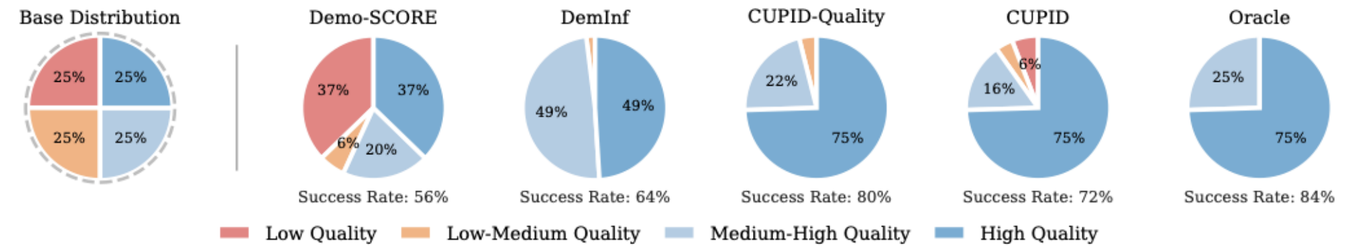
Figure-8: Improving Policy Performance in Mixed-Quality Regimes

TuckBox: Identifying Robust Test-Time Strategies from Policy Failures (heterogeneous datasets)

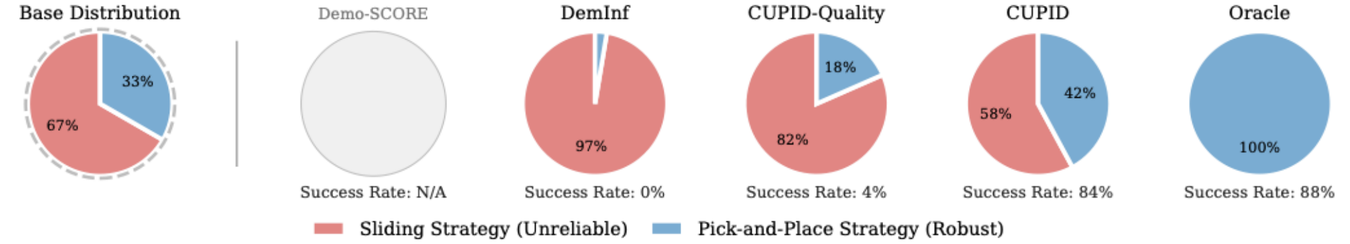
Bookshelf: Disentangling Spurious Correlations in Demonstration Data

1. Demo-score failed in tuckbox because sliding is less variant.
2. Demo-score misattributed failure in bookshelf because of the failure arm state and book state.

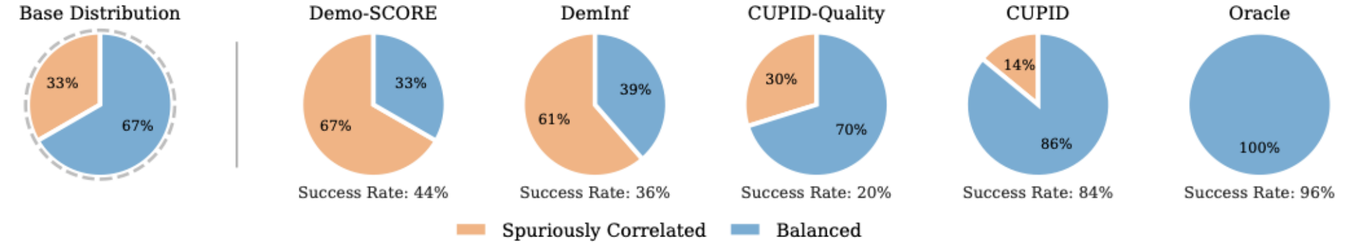
Data composition:



(a) **Figure-8:** Distribution of curated demonstrations after *filtering* 66%. Higher-quality demos are better.



(b) **TuckBox:** Distribution of curated demonstrations after *filtering* 66%. Pick-and-place demos are better.



(c) **Bookshelf:** Distribution of curated demonstrations after *filtering* 50%. Balanced data is better.

VLA Robustness. Fig. 7 also suggests that scaling the pre-training of VLA models does not inherently enable them to leverage their generalist knowledge to, e.g., *ignore* low-quality behaviors or brittle strategies in demonstration data. That is, data curation still appears important for VLA post-training.

On pi 0

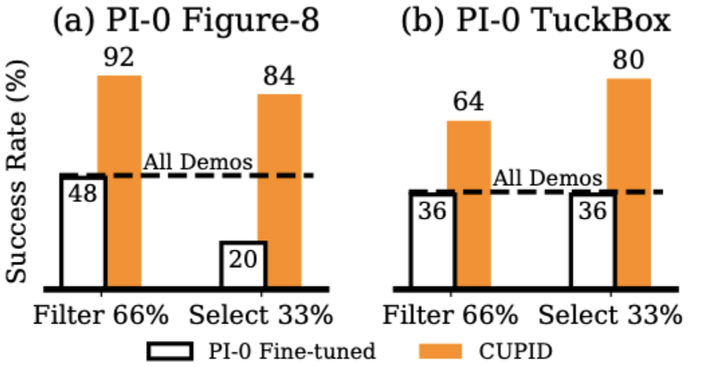


Figure 7: Data curated for single-task diffusion policies improves π_0 [21] post-training performance. Additional results in Fig. 15.

Limitations:

1. Curation tasks are fixed sized.
2. Unclear when curation helps most.(train another NN to find hidden features?)
3. Poor at data explainability(for data collectors).(Design an engineering workline)
4. Ignores interaction among demos.(use Shaplely?)
5. Expensive on large datasets.
6. High variance due to REINFORCE estimator, requires many rollouts.

(Optimize rollouts?)

Basu et al. (2021) — *Influence Functions in Deep Learning Are Fragile*

Applications

1. Influence functions could be used on other other data processing techniques (like used on pi0.7)

2. Quality check are mostly based on heuristics(pi 0.7,robomimic MH dataset), which may be biased and not true, CUPID could work as a casual verifier.

3. **Reward models** could replace the influence function(f)(but double estimation)

$$\Psi_{\text{inf}}(\hat{z},z) := \left.\frac{df(\hat{z};\theta)}{d\epsilon}\right|_{\epsilon=0} = -\nabla_\theta f(\hat{z};\theta(\mathcal{D}))^\top H_\theta^{-1} \nabla_\theta \ell(z;\theta(\mathcal{D})),$$

What I’m currently doing on this project:

In the near future:

1. Implement the method on galbot clothes folding/garbage collection dataset.

2. Experiment with some other dataset process techniques(deduplication, reweighting) on datasets to enhance model behaviors.

3. Explore the broad techniques in LLM on influence functions and large-data processing methods and implement them in robotics

What I’m currently doing on this project:

In the long run:

Robotic generalization should not rely on the quality of datasets, improving the model’s capacity is more important than curating datasets, however the model’s way of absorbing data could learn from current data-curating techniques.

Overall Summary:

1. Improved ai using skills.
2. Better at planning and documenting.
3. A deep experience of life of pure researching.
4. Communication with senior and peers.